

Lab #4

CS744 Autumn 2026

c f l a s k

This exercise is inspired by Flask and FastAPI – python-based web application frameworks. For more info: [link1](#) [link2](#) [link3](#) [link4](#)

Example #1

The following code block (`helloworld.py`) demonstrates a simple Flask example. It exposes the `helloworld()` function to the web and can be accessed via an HTTP URL. Flask, by default, spins a server on localhost listening for requests on port 5000 (default port number, but configurable).

Python

```
# helloworld.py
# Importing flask module in the project is mandatory
# An object of Flask class is our WSGI application.
from flask import Flask

# Flask constructor takes the name of
# current module (__name__) as argument.
app = Flask(__name__)

# The route() function of the Flask class is a decorator,
# which tells the application which URL should call
# the associated function.
@app.route('/')
# '/' URL is bound to the hello_world() function.
def helloworld():
    return 'Hello, World!'

# main driver function
if __name__ == '__main__':

    # run() method of Flask class runs the application
    # on the local development server.
    app.run()
```

Sample Output:

```
Shell
$ python helloworld.py
* Serving Flask app 'helloworld'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit

// try this in another terminal window, or access URL in browser
$ curl http://127.0.0.1:5000
Hello, World!
```

Example #2

The following code block demonstrates how to deal with GET query parameters. The code snippet below can parse a URL similar to */hello?name=BuzzLightYear* and store *BuzzLightYear* in the variable *name*.

```
Python
# hello.py
from flask import Flask, request
app = Flask(__name__)

@app.route('/hello')
def helloname():
    name = request.args.get('name')
    return 'Hello %s!' % name

if __name__ == '__main__':
    app.run()
```

Sample Output:

```
Shell
$ python3 hello.py
* Serving Flask app 'hello'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment.
Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

```
// try this in another terminal window, or access URL in browser
$ curl "http://127.0.0.1:5000/hello?name=BuzzLightYear"
Hello BuzzLightYear!
```

In this assignment, we will recreate Flask-like/FastAPI-like functionality — adding new functions and exposing them via a URL.

The first task is to implement a web server using the Linux sockets API and POSIX threads with the C-programming language.

To keep the focus on server design rather than text processing, we provide a small HTTP parser ([http-parser.h](#), [http-parser.c](#)) that processes an HTTP request into its components and builds HTTP responses based on response data and control messages.

Some **references** to the HTTP protocol and socket programming are — [Link1](#) [Link2](#) [Link3](#) **Appendix A**

Anatomy of a URL

Shell

```
http://127.0.0.1:8080/arithmetic/square?num=3#result
__/  \_____/  \_/\_____/\____/  \____/
scheme  host    port    path      query  fragment
```

Anatomy of an HTTP Request

A HTTP request has four parts – a request line, header lines, one empty line, and an optional body. Every line ends with the two bytes `\r\n` (carriage return, line feed), written CRLF.

Part 0.0 (Flask Setup and Usage)

Shell

```
$ python3 -m venv flaskenv  
$ source flaskenv/bin/activate  
$ pip install flask
```

Save Example #1 as `helloworld.py` and run it:

Shell

```
$ python3 helloworld.py  
$ curl http://127.0.0.1:5000/
```

Example #2 (`hello.py`)

Shell

```
$ python3 hello.py  
$ curl "http://127.0.0.1:5000/hello?name=BuzzLightYear"
```

Seeing the HTTP request

Terminal 1 (Server)

Shell

```
$ python3 hello.py
```

Terminal 2 (Client)

send the request with `-v` (verbose), so curl also prints the request it sends and the response headers it receives:

None

```
$ curl -v "http://127.0.0.1:5000/hello?name=BuzzLightYear"  
* Trying 127.0.0.1:5000...  
* Established connection to 127.0.0.1 (127.0.0.1 port 5000) from 127.0.0.1 port 40394  
* using HTTP/1.x
```

```
> GET /hello?name=BuzzLightYear HTTP/1.1
> Host: 127.0.0.1:5000
> User-Agent: curl/8.18.0
> Accept: */*
>
* Request completely sent off
< HTTP/1.1 200 OK
< Server: Werkzeug/3.1.9 Python/3.14.4
< Date: Fri, 02 Oct 2026 09:18:53 GMT
< Content-Type: text/html; charset=utf-8
< Content-Length: 20
< Connection: close
<
* shutting down connection #0
Hello BuzzLightYear!
```

- Lines starting with **>** are the **request** curl sent.
- Lines starting with **<** are the **response** headers from the server.
- Lines starting with ***** are curl's own notes about the connection.
- The last line is the response body.

Stop the server with Ctrl+C. Run **deactivate** to leave the virtual environment.

To observe –

1. how a URL is mapped to a function (**@app.route**)
2. how query parameters reach the function (**request.args**), and that Flask chooses a port (5000) and serves requests until stopped.

The goal of **cflask** is to implement these ideas in C.

Part 0.1 (Starter Code Walkthrough)

File	Purpose
<code>http-parser.h</code>	Structs (ParsedRequest, HttpRequestLine, HttpHeaders) and function prototypes
<code>http-parser.c</code>	<code>parse_http_request()</code> and <code>create_http_response()</code>
<code>cflask.c</code>	Skeleton for your server

Building a server: the socket API

`man 2 socket`, `man 2 bind`, `man 2 listen`, `man 2 accept`,
`man 2 recv`, `man 2 send`

The **server** program has the following sequence –

1. create a socket and get socket fd (via the socket system call)
2. bind socket to local host's port (via the bind system call)
3. listen for incoming requests and be ready to accept connections
4. accept an incoming TCP connection request and get a new socket fd for the connection
5. read (HTTP) message (request) from the client
6. parse HTTP message and extract information in the URL
7. dispatch to appropriate function based on URL
8. generate and send HTTP response
9. close connection
10. with a single-threaded server, jump back to step 4

Part 1

Part 1a: Single-threaded cflask

Build a server that handles one request at a time: accept a connection, serve it completely, close it, then accept the next.

Usage:

Shell

```
./cflask s <port>
```

Argument	Meaning
s	Single threaded mode
<port>	TCP port on which the server accepts requests

Note: If the arguments are missing or invalid, print a usage message and exit.

In addition to the provided starter files ([http-parser.h](#), [http-parser.c](#)) —

- I. **functions.h** lists every URL exposed by the server and assigns each URL an integer ID. The ID is the index used to find the function for that URL. Write the mapping as valid C, for example as an array of `{url, id}` pairs or as `#define` constants. Conceptually, the mapping looks like:

None

```
/          0
/square    1
/cube      2
/helloworld 3
/pingpong  4
```

- II. **functions.c** contains the implementations of all functions exposed on the web. All functions must have the same signature, so that they can be stored in one function pointer array.

- III. **functionslst.h** contains a function pointer array that places each function from **functions.c** at the index given by its ID in **functions.h**.
- IV. **cflask.c** contains the server. It creates the listening socket, accepts connections, receives and parses each request, looks up the requested URL, calls the matching function, and sends the response. Requests for URLs not listed in **functions.h** get a **404 Not Found** response.

Note: in this part, all request URLs have a single path segment (e.g. **http://127.0.0.1:8080/square**) and contain no query parameters.

Terminal 1 (Server)

```
Shell
$ ./bin/cflask s 8080
cflask [single-threaded] on http://0.0.0.0:8080
[thread 22122] GET /square -> 200 OK
[thread 22122] GET /helloworld -> 200 OK
[thread 22122] GET /nosuchpath -> 404 Not Found
```

Terminal 2 (Client)

```
Shell
$ curl http://127.0.0.1:8080/square
1

$ curl http://127.0.0.1:8080/helloworld
Hello

$ curl http://127.0.0.1:8080/nosuchpath
Not Found
```

Part 1b: Multi-threaded cflask

Extend the server so that every request is served by its own thread. The main thread only accepts connections. For each new connection, it creates a worker thread and passes it the connected socket. The worker serves that one request, closes the socket, and exits.

Sequence for the thread-per-request server

The work is split between two kinds of threads. The main thread that only accepts connections. and worker threads that are created on each request serve the request and finish.

Main thread –

1. create a socket and get a socket fd (via the socket system call)
2. bind the socket to the local host's port (via the bind system call)
3. listen for incoming requests and be ready to accept connections
4. accept an incoming TCP connection request and get a new socket fd for the connection
5. create a new worker thread and pass it the new socket fd
6. jump back to step 4 (do not wait for the worker thread to finish)

Each worker thread —

1. read the HTTP message (request) from the client
2. parse the HTTP message and extract information in the URL
3. dispatch to the appropriate function based on the URL
4. generate and send the HTTP response
5. close the connection
6. exit the thread

Usage:

Shell

```
./cflask m <port>
```

Argument	Meaning
m	Thread per Request Mode
<port>	TCP port on which the server accepts requests

Terminal 1 (Server)

Shell

```
$ ./bin/cflask m 8080
cflask [thread per request] on http://0.0.0.0:8080
[thread 21428] GET /square -> 200 OK
[thread 21519] GET /helloworld -> 200 OK
```

```
[thread 21580] GET /square -> 200 OK  
[thread 21657] GET /nosuchpath -> 404 Not Found
```

Terminal 2 (Client)

```
Shell  
$ curl http://127.0.0.1:8080/square  
1  
  
$ curl http://127.0.0.1:8080/helloworld  
Hello  
  
$ curl http://127.0.0.1:8080/square  
1  
  
$ curl http://127.0.0.1:8080/nosuchpath  
Not Found
```

Part 1c: Thread-pool cflask

Extend the server with a fixed **pool** of worker threads that is created once, at startup. The main thread only accepts connections and places each client socket in a shared queue. Each worker thread repeatedly takes a socket from the queue, serves the request, and closes the socket. Worker threads never exit.

Sequence for Thread Pool Server

The server program has the following sequence:

Main thread:

1. create a socket and get a socket fd (via the socket system call)
2. bind the socket to the local host's port (via the bind system call)
3. listen for incoming requests and be ready to accept connections
4. create a fixed number of worker threads (the thread pool)
5. accept an incoming TCP connection request and get a new socket fd for the connection
6. add the new socket fd to a shared queue
(if the queue is full, wait until a worker removes one)
7. jump back to step 5

Each worker thread:

1. pick a socket fd from the shared queue (if the queue is empty, wait until the main thread adds one)
2. read the HTTP message (request) from the client
3. parse the HTTP message and extract information in the URL
4. dispatch to the appropriate function based on the URL
5. generate and send the HTTP response
6. close the connection
7. jump back to step 1 (the worker thread never exits)

Usage:

Shell

```
./cflask t <port> <num_threads>
```

Argument	Meaning
t	Thread Pool Mode
<port>	TCP port on which the server accepts requests
<num_threads>	Number of worker threads in the pool

Terminal 1 (Server)

Shell

```
$ ./bin/cflask t 8080 2
cflask [thread pool, 2 threads] on http://0.0.0.0:8080
[thread 21805] GET /nosuchpath -> 404 Not Found
[thread 21806] GET /square -> 200 OK
[thread 21805] GET /helloworld -> 200 OK
[thread 21806] GET /helloworld -> 200 OK
[thread 21805] GET /square -> 200 OK
```

Terminal 2 (Client)

```
Shell
$ curl http://127.0.0.1:8080/nosuchpath
Not Found

$ curl http://127.0.0.1:8080/square
1

$ curl http://127.0.0.1:8080/helloworld
Hello

$ curl http://127.0.0.1:8080/helloworld
Hello

$ curl http://127.0.0.1:8080/square
1
```

Part 2 : Dealing with HTTP request arguments

Extend the server to pass arguments to the functions. Arguments can arrive in either of two places, and you may support either one or both:

- **GET** in the query string after `?`, e.g. `/square?num=3`
- **POST** in the request body, encoded the same way (application/x-www-form-urlencoded), e.g. `curl -d "num=3" http://127.0.0.1:8080/square`

Note: In this part, URLs still have a single path segment.

Argument format

1. Arguments are **key=value pairs** separated by `&`.
2. Your parser must find the argument by name, regardless of its position.

```
Shell
/square?unit=cm&num=4
```

```
# query parameters:
#   unit = cm
#   num  = 4
```

In the second part of the assignment, extend the implementation to handle arguments passed in the URL as either query parameters in a GET request or inside a request body of a POST request. You can choose any of the above techniques to handle HTTP arguments.

HTTP requests will require careful parsing to extract the arguments from the request and then be supplied to the function corresponding to the HTTP request.

Terminal 1 (Server)

```
Shell
$ ./bin/cflask t 8080 4
cflask [thread pool, 4 threads] on http://0.0.0.0:8080
[thread 22943] GET /square -> 200 OK
[thread 22944] GET /cube -> 200 OK
[thread 22945] GET /helloworld -> 200 OK
```

Terminal 2 (Client)

```
Shell
$ curl "http://127.0.0.1:8080/square?num=3"
9

$ curl "http://127.0.0.1:8080/cube?num=7"
343

$ curl "http://127.0.0.1:8080/helloworld?str=Ashwin"
Hello, Ashwin
```

Part 3 : Two-segment URLs (optional)

Extend URL handling to paths with two segments, such as:

```
None
http://127.0.0.1:8080/arithmetic/fibonacci?num=10
```

Add the new URLs to `functions.h` like any other entry:

```
None
/arithmetic/prime      5
/arithmetic/fibonacci  6
```

Output:

```
Shell
$ curl "http://127.0.0.1:8080/arithmetic/prime?num=13"
True

$ curl "http://127.0.0.1:8080/arithmetic/prime?num=15"
False

$ curl "http://127.0.0.1:8080/arithmetic/prime"
False

$ curl "http://127.0.0.1:8080/arithmetic/fibonacci?num=10"
55

$ curl "http://127.0.0.1:8080/arithmetic"
Not Found
```

Part 4 : (Optional)

Build an in-memory representation (a search tree, or a hash-based store) to store the URL to function ID mappings ... instead of reading this information from a file on each HTTP request.

Part 5 : Functions

Implement the following functions. They are used to test Parts 1–3.

1. `/`
This just returns a hello world message back to the user.
This URL will not have any arguments.
2. `/square`
This URL requests computing the square of a number which can be passed along with the URL as a query parameter like `/square?num=3` (return the square of 3). If no query parameter is present then this function should return the value 1.
3. `/cube`
This URL requests computing the cube of a number which can be passed along with the URL as a query parameter like `/cube?num=7` (return the cube of 7). If no query parameter is present then this function should return the value 1.
4. `/helloworld`
This URL maps to a function that takes a string argument as a query parameter, and returns the same string prefixed by hello back to the user. For example, `/helloworld?str=Ashwin`, returns back **“Hello, Ashwin”**. In case of no query parameter, it simply returns **“Hello”** back to the user.
5. `/pingpong`
This URL maps to a function that takes a string argument as a query parameter, and returns the same string back to the user. For example, `/pingpong?str=cs744`, returns back **“cs744”**. In case of no query parameter, it simply returns **“PingPong”** back to the user.
6. `/arithmetic/prime`
This URL maps to a function that takes a number as a query parameter and returns **‘True’** to the user if the number is a prime and **‘False’** if the number is not prime. In case of no query parameter, the function returns **‘False’** back to the user.
7. `/arithmetic/fibonacci`
This URL maps to a function that takes a number **k** as a query parameter and returns **the kth fibonacci number** to the user. In case of no query parameter, the function returns **1** back to the user.

Part 6 : Load Testing

Measure the throughput and response time variation with changing load for each server mode. Load is the number of requests hitting the server in a fixed duration, i.e., the request rate and/or the number of connection requests to the server etc.

1. **Setup.** Pin the server and `wrk` to separate CPU cores, as described in the "Pinning the server and the client" section below.
2. **Load.** Run each measurement for a fixed duration (e.g. `-d 30s`), and vary the number of concurrent connections (`-c`), for example 1, 2, 4, 8, 16, 32, 64, 128
3. **Server configurations.** For each function you test, measure:
 - single-threaded (`./cflask s <port>`),
 - thread per request (`./cflask m <port>`),
 - a thread pool with at least three different sizes (e.g., `./cflask t <port> 1, 2, 4, 8`).
4. **Functions.** Repeat the test for at least three functions. At least one must be CPU-heavy, for example `/arithmetic/prime?num=1000000007`.
5. **Plots.** For each function, make two graphs, with one line per server configuration:
 - throughput (requests/sec) vs. concurrent requests,
 - mean response time (ms) vs. concurrent requests.Label both axes with units. Use **Requests per second** and the first **Time per request (mean)** line from `wrk` 's output.

Hypothesis: Throughput increases with load and then flattens once the server's CPUs are saturated. After that point, response time grows roughly in proportion to the number of concurrent requests.

You can use the **wrk tool** to load test the cflask implementation.

<https://github.com/wg/wrk>

Experiment: single-threaded vs on-demand multi-threaded vs thread pool

Aim: Measure how throughput and mean response time change with load for each server design, on the same fixed set of CPU cores.

Hypothesis:

1. **Throughput rises with load, then flattens.** At low concurrency the server is often idle, waiting for the next request, so adding connections increases throughput. Once the server's CPU cores are fully busy, extra connections cannot make it serve requests any faster, and throughput stops rising.
2. **After that point, response time grows with load.** Extra requests just wait their turn, so mean response time increases roughly in proportion to the number of concurrent connections (Little's law: response time $\approx$ connections $\div$ throughput).

Configuration	Command
Single Threaded (Part 1a)	<code>./cflask s 8080</code>
On-demand multi-threaded (Part 1b)	<code>./cflask m 8080</code>
thread pool of 1, 2, 4, 8 (Part 1c)	<code>./cflask t 8080 1 (then 2, 4, 8)</code>

For each configuration, you vary concurrency (`wrk -c`) over 1, 2, 4, 8, 16, 32, 64, 128 and record throughput and mean response time.

Step 1: Find your CPU layout

Shell

```
$ lscpu -e
```

CPU	NODE	SOCKET	CORE	L1d:L1i:L2:L3	ONLINE	MAXMHZ	MINMHZ	MHZ
0	0	0	0	0:0:0:0	yes	4673.0000	400.0000	3768.1221
1	0	0	1	1:1:1:0	yes	4673.0000	400.0000	4371.9868
2	0	0	2	2:2:2:0	yes	4673.0000	400.0000	3945.6050
3	0	0	3	3:3:3:0	yes	4673.0000	400.0000	3764.2400
4	0	0	4	4:4:4:0	yes	4673.0000	400.0000	4378.7178
5	0	0	5	5:5:5:0	yes	4673.0000	400.0000	400.0000
6	0	0	6	6:6:6:0	yes	4673.0000	400.0000	3723.7639
7	0	0	7	7:7:7:0	yes	4673.0000	400.0000	400.0000
8	0	0	0	0:0:0:0	yes	4673.0000	400.0000	400.0000

9	0	0	1 1:1:1:0	yes	4673.0000	400.0000	400.0000
10	0	0	2 2:2:2:0	yes	4673.0000	400.0000	400.0000
11	0	0	3 3:3:3:0	yes	4673.0000	400.0000	400.0000
12	0	0	4 4:4:4:0	yes	4673.0000	400.0000	3691.4951
13	0	0	5 5:5:5:0	yes	4673.0000	400.0000	3689.3330
14	0	0	6 6:6:6:0	yes	4673.0000	400.0000	400.0000
15	0	0	7 7:7:7:0	yes	4673.0000	400.0000	400.0000

Note: The **CORE** column tells you which logical CPUs share a physical core. This machine has 8 physical cores (0–7) with 2 hyperthreads each.

Step 2: Choose the core assignment

The server (**cflask**) and the load generator client (**wrk**) must run on different physical cores. Hyperthread siblings share a core's execution units, so avoid them too. Otherwise the server's two threads would compete with each other, or with **wrk**, for the same core.

On this machine:

Application	Logical CPU	Physical Cores
cflask (server)	0,1	0,1
wrk (client)	4,5,6,7	4,5,6,7

Note: You can adjust core assignment according to your system

Step 3: Start the server, pinned and run a sample measurement

In Terminal 1:

```
Shell
$ sudo taskset -c 0,1 ./cflask s 8080
cflask [single-threaded] on http://0.0.0.0:8080
```

In Terminal 2:

Shell

```
$ sudo taskset -c 4-7 wrk -t4 -c32 -d30s -H "Connection: close"
"http://127.0.0.1:8080/arithmetic/prime?num=1000000007"
```

Sample Output

Shell

```
$ taskset -c 4-7 wrk -t4 -c32 -d30s -H "Connection: close"
"http://127.0.0.1:8080/arithmetic/prime?num=1000000007"
Running 30s test @ http://127.0.0.1:8080/arithmetic/prime?num=1000000007
  4 threads and 32 connections
  Thread Stats   Avg      Stdev     Max    +/-  Stdev
    Latency    1.22ms   52.59us   2.23ms   89.50%
    Req/Sec    6.38k    296.87    6.53k    96.17%
  762765 requests in 30.10s, 63.29MB read
Requests/sec: 25340.30
Transfer/sec:  2.10MB
```

- **-c32 is the load.** `wrk` keeps 32 connections busy at all times, so 32 requests are in flight at once.
- **-t4** is how many client threads `wrk` uses to simulate those users.
- Every thread needs at least one connection to work with, so **-t** must not be larger than **-c**.

What to record:

None

Requests/sec is your **throughput**.

Latency Avg is your **mean response time**. Its unit changes with the value (us, ms, s), so convert everything to milliseconds when you record it.

Req/Sec is **per client thread**, not the total. Do not use it as throughput.

Note:

While a run is going, execute `top` or `htop` in a third terminal and press `1` to see each CPU:

- **CPUs 0 and 1** (server) should be near 100% once the load is high enough.
- **CPU 4,5,6,7** (`wrk`) should **not** be at 100%. If it is, you are measuring `wrk`, not your server.

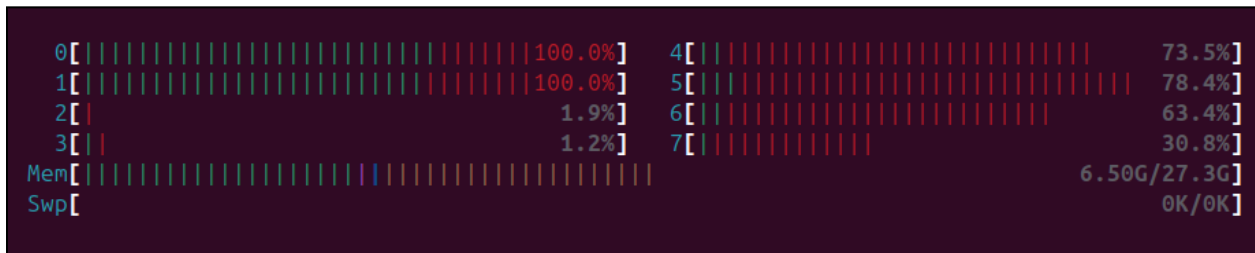


Fig 1. Showing server CPUs (0,1) are saturated

Step 4 : Sweep the concurrency levels

You can use a similar command to do so or create a script.

Shell

```
for c in 1 2 4 8 16 32 64 128; do
    t=$(( c < 4 ? c : 4 ))      # wrk needs -t <= -c
    echo "== concurrency $c"
    taskset -c 4-7 wrk -t$t -c$c -d30s -H "Connection: close" \
        "http://127.0.0.1:8080/arithmetic/prime?num=1000000007" \
        | grep -E "Latency|Requests/sec|Socket errors|Non-2xx"
done | tee raw_s.txt
```

Step 5 : Repeat for every configuration and function

Stop the server in Terminal 1 with `Ctrl+C`, then start the next configuration with the same `taskset -c 0,1`. For each one, repeat the above steps or create an automation script.

Step 6 : Visualization

For each function, draw one figure with two graphs, with one line per server configuration (s , m , $t1$, $t2$, $t4$, $t8$):

- **Throughput** (requests/sec) vs. concurrent connections (`wrk -c`)
- **Mean response time** (ms) vs. concurrent connections (`wrk -c`)

Label both axes with units, and start all y-axes at 0. Because the number of connections doubles at each step, use a base-2 log scale on the x-axis.

Below is a **sample plot from our reference implementation**, with the server on CPUs 0–1 and `wrk -t4` on CPUs 4–7, 30 s per point. Your numbers will differ, but the shapes should be similar. Compare the configurations: which one flattens lowest, which highest, and do pools larger than the number of server cores help?

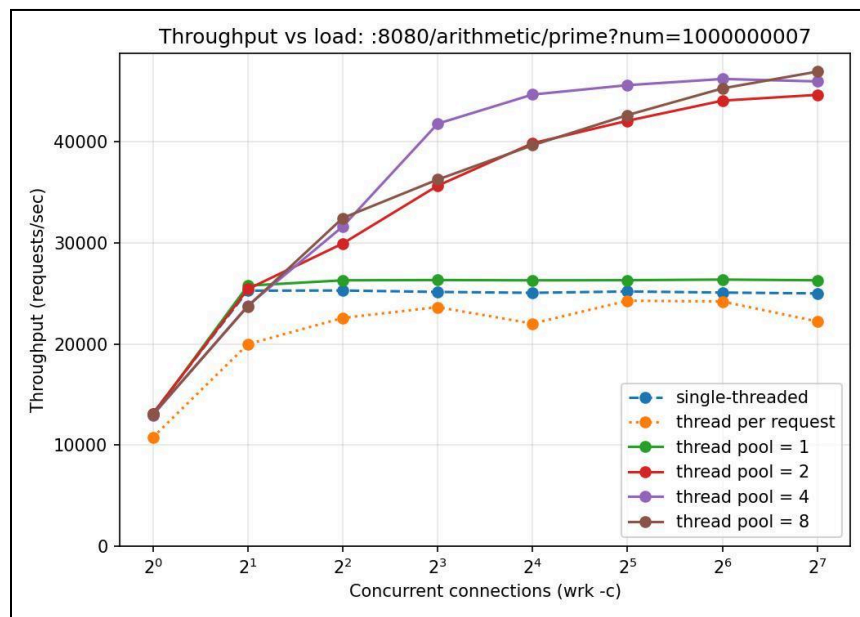


Fig 2a: Throughput vs. concurrent connections for `/arithmetic/prime?num=1000000007`. Server pinned to CPUs 0–1, `wrk -t4` on CPUs 4–7, 30 s per point.

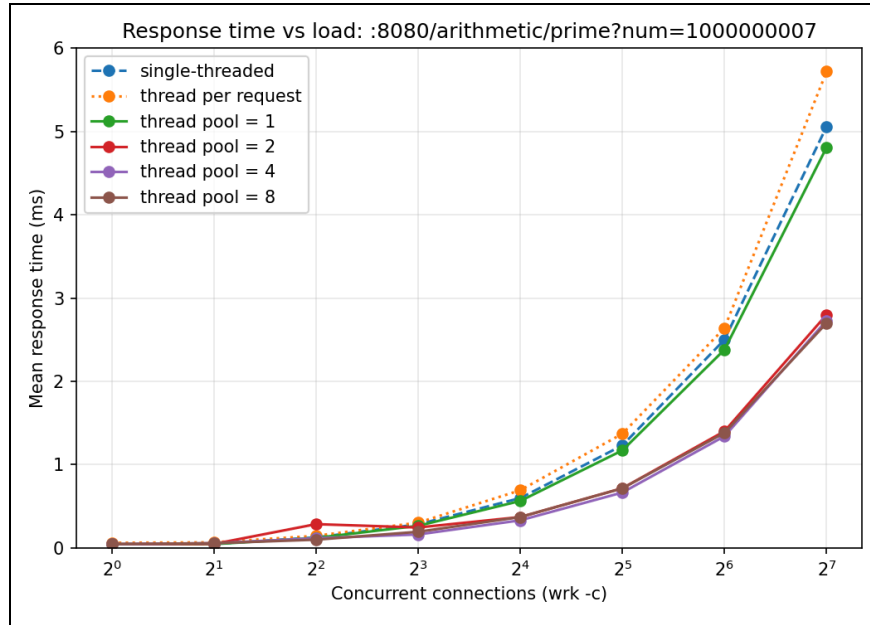


Fig 2b: Mean response time vs. concurrent connections

Observations

The figure above shows the results for `/arithmetic/prime?num=1000000007`, with the server pinned to two cores (CPUs 0–1) and `wrk -t4` on four other cores (CPUs 4–7), 30 seconds per point. Every run completed with no errors.

Configuration	Throughput at High Load	Compared with single-threaded
Single-threaded	~25,100 req/s	—
Thread per request	~22,000–24,000 req/s	slightly lower
Thread pool = 1	~26,300 req/s	about the same
Thread pool = 2	~44,700 req/s	about 1.8×
Thread pool = 4	~46,000 req/s	about 1.8×
Thread pool = 8	~47,000 req/s	about 1.9×

1. Throughput rises, then flattens.

With one connection (`wrk -c`), every configuration serves about 13,000 req/s: there is only one request at a time, so extra threads have nothing to do. As connections increase, throughput rises until the server's CPUs are fully busy, and then it stays flat. Adding more load after that point does not produce more completed requests.

2. The single-threaded server uses only one core.

It flattens at about 25,000 req/s, already at 2 connections. One thread can run on only one CPU at a time, so the second core stays idle. This also tells us the cost of one request on this machine: $1 \div 25,000 \approx 40 \mu\text{s}$, including accepting the connection, reading, parsing, the prime check, sending and closing.

3. A thread pool with 2 or more workers uses both cores.

Pools of 2, 4 and 8 reach about 45,000–47,000 req/s, almost twice the single-threaded server. It is not exactly 2× because each request also involves work that does not split cleanly between cores: kernel networking (connect, send, close), the main thread that accepts connections, and the hand-off of each socket through the shared queue.

4. Workers beyond the number of cores do not help.

Pools of 4 and 8 end up close to the pool of 2. The server has only two cores, so at most two threads can run at any moment; the others wait their turn. **Threads do not add CPU capacity, cores do.**

5. Thread per request is no faster than single-threaded, even though it creates many threads.

For every request, the main thread must call `accept()` and `pthread_create()`. Creating a thread means the kernel allocates a stack, sets up a new task and schedules it; when the thread exits, its stack is freed again.

7. Response time grows in proportion to load.

Once a configuration is saturated, its mean response time increases roughly linearly with the number of connections, because extra requests just wait in a queue.

Reporting of experiments

Use `wrk` as the default load generator. You may also try other load generators you know or want to explore (for example `ab`).

For **every** experiment, your report must include:

1. **Aim:** what question the experiment answers.
Example: how does throughput change with load for each server design?
2. **Setup and execution:**
 - the machine (CPU model, number of cores) and the core assignment (`taskset` values for the server and the load generator);
 - the exact commands used to start the server and run the load generator;

- the **independent parameter**, i.e. what you varied (e.g. number of concurrent connections, `wrk -c`);
 - the **metrics**, i.e. what you measured (e.g. throughput in requests/sec, mean response time in ms).
3. **Hypothesis**: what you expect to happen, and why, written **before** you run the experiment.
 4. **Observations**: what the data and plots actually show, described with numbers. For example: where throughput flattens, at what value, and how the configurations compare.
 5. **Explanation and inferences**: why the system behaves this way. Did the results match your hypothesis? If not, why not?

Repeat this for each experiment, then add one more section that **reasons across all your experiments together**: what do the different functions and configurations show when compared side by side, and what general conclusions can you draw about server design?

Submission guidelines

Put all files in a directory named `<rollno>-cs744-lab4`, using your roll number, e.g. `25M0001-cs744-lab4`:

```
<rollno>-cs744-lab4
├── cflask.c           server: modes s, m, t (Parts 1-3)
├── functions.h        URL -> function ID mapping
├── functions.c        function implementations
├── functionslist.h    function pointer array
├── http-parser.h      provided parser (unchanged)
├── http-parser.c      provided parser (unchanged)
├── scripts/          load-generation and plotting scripts you used
├── plots/
│   ├── loadtest1.data results + exact server and wrk commands used
│   ├── loadtest1.jpg  throughput and response-time graphs
│   ├── loadtest2.data
│   ├── loadtest2.jpg
│   ├── loadtest3.data
│   └── loadtest3.jpg
```

Upload the zipped file `<rollno-cs744-a1>.tar.gz` of the submission folder on Moodle.

Submission deadline: 16th October 2026, 11.59 pm

Appendix

Appendix A: HTTP Protocol Primer

A.1 Structure of an HTTP request

An HTTP request is plain text with four parts, in this order:

1. a request line
2. header lines, one per line
3. an empty line
4. a body (optional)

Every line ends with the two characters `\r\n` (carriage return, line feed). For example, `curl "http://127.0.0.1:8080/square?num=3"` sends:

```
None
GET /square?num=3 HTTP/1.1\r\n      <- request line
Host: 127.0.0.1:8080\r\n          <- headers
User-Agent: curl/8.5.0\r\n
Accept: */*\r\n
\r\n                                <- empty line: headers end here
```

A.2 The request line

The request line has three parts separated by spaces: `METHOD PATH VERSION`, for example `GET /square?num=3 HTTP/1.1`.

- The **path** includes the query string (everything after `?`).

Method	Meaning	Body
GET	Fetch a resource; arguments go in the query string	No
POST	Send data to the server; arguments go in the body	Yes
PUT, DELETE, ...	Other operations	Varies

A.3 Headers

Each header is **Name: value**. Names are case-insensitive, so **Content-Length** and **content-length** are the same header. The ones you will meet:

Header	Example	Meaning
Host	127.0.0.1:8080	Server name and port
Content-Length	5	Size of body in bytes
Content-Type	<code>application/x-www-form-urlencoded</code>	Format of the body
Connection	Close or keep alive	Whether the TCP connection stays open after the response
User-Agent	ApacheBench/2.3	Which client sent the request

A.4 Structure of an HTTP response

A response has : a status line, headers, an empty line, and the body. For example, cflask's reply to `/square?num=3`:

```
None
HTTP/1.1 200 OK\r\n          <- status line
Content-Type: text/plain\r\n  <- headers
Content-Length: 1\r\n
Connection: close\r\n
\r\n                          <- empty line
9                             <- body
```

A.5 References

- **Kurose & Ross, Computer Networking: A Top-Down Approach, Chapter 2 videos:**
https://gaia.cs.umass.edu/kurose_ross/videos/2/
- **MDN, An overview of HTTP:**
<https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Overview>
A readable introduction to clients, servers and the request/response flow.

- **Socket Programming (Dartmouth CS 50, Lecture 19):**
<https://www.cs.dartmouth.edu/~campbell/cs50/socketprogramming.html>
The TCP socket API (`socket`, `bind`, `listen`, `accept`, `send`, `recv`, `close`), with a complete echo server example.
- **RFC 9112, HTTP/1.1:** <https://www.rfc-editor.org/rfc/rfc9112>